

# Resolution Report

## BUGS AND ERRORS FIXED IN MIS

This document responds to the test report of Friday, 14th August 2026 (*Bugs and Errors Found in MIS*). Every issue raised in that report has been investigated, corrected and verified. Items are numbered to match the original report exactly, so the two documents can be read side by side.

**Outcome:** all 9 reported issues resolved.

Five root causes proved to be different from what the symptoms suggested - in three cases the reported feature could never have worked for any user, and in one case (issue 8) the data was correct and only its presentation was at fault. Fixes were verified by reproducing each fault first, then re-testing against the running system and the live database.

| # | ISSUE                                                                  | STATUS |
|---|------------------------------------------------------------------------|--------|
| 1 | Changing/Adding a profile picture sends a 401 storage Error            | Fixed  |
| 2 | Global consistent information (structured location data)               | Fixed  |
| 3 | Notification feature in header doesn't work                            | Fixed  |
| 4 | During Application process users can skip required documents           | Fixed  |
| 5 | Changing password within student portal sends an internal server error | Fixed  |
| 6 | Uploading National ID or Passport fails                                | Fixed  |
| 7 | Fine payment                                                           | Fixed  |
| 8 | Fake payment calendar / unclear calendar                               | Fixed  |
| 9 | Application payment bypass (transcript not visible to admin)           | Fixed  |

## Student Portal - Issues Resolved

## 1. Changing/Adding a profile picture sends a 401 storage Error

### REPORTED

On the student profile page, changing or adding a profile picture returned "File storage service error (401): Unauthorized. Invalid API Key.", with a 422 (Unprocessable Entity) in the console.

### ROOT CAUSE

The backend authenticates to the standalone file-storage service with an **X-API-Key** header taken from **FILE\_SERVER\_KEY**. That key was empty or did not match the value in **file-server/.env**, so storage answered 401. The backend then re-threw that upstream message to the browser as a **422 validation error** - telling the student their photo was invalid, when nothing was wrong with the file at all.

### WHAT WAS CHANGED

- a. Introduced **FileServerException**, which separates a genuine file problem (too large or wrong type, answered with **422**) from a storage or configuration failure (answered with **502 Bad Gateway**).
- b. The storage client now validates **FILE\_SERVER\_URL** and **FILE\_SERVER\_KEY** before sending anything, instead of transmitting a blank key and receiving a cryptic 401.
- c. Upstream technical detail now goes to the server error log - including the exact remedy ("FILE\_SERVER\_KEY in backend/.env does not match file-server/.env") - while the user sees an honest "storage service is unavailable" message.
- d. Profile-photo endpoints no longer reject an upload because the browser reported a non-standard MIME type such as *image/jpg*; the authoritative check is content-based (finfo), which is also the more secure test.
- e. Added **GET /api/health?deep=1**, which reports whether storage is configured and reachable, so this class of fault can be diagnosed without reproducing a failed upload.

**Verification.** Reproduced the exact 401 against the running file server. It now returns 502 with a user-appropriate message, and the log carries the precise cause and fix. Genuine file errors (oversized, wrong content type) still correctly return 422. All five failure modes tested: correct key, wrong key, missing key, missing URL, unreachable service.

## 2. Global consistent information (structured location data)

### REPORTED

Application forms required users to type their residence details by hand. The request was for cascading dropdowns - Province, District, Sector, Cell, Village - based on accurate predefined administrative data, where each selection filters the next level.

### ROOT CAUSE

No structured location data existed anywhere in the system. Province was a small hardcoded list and every level below it was free text, so nothing constrained or validated what users entered.

### WHAT WAS CHANGED

- a. Added **rwandaLocations.ts** holding the 5 provinces and all 30 districts, each district mapped to its province.
- b. Added a reusable cascading **LocationSelect** component. Choosing a province narrows the district list to that province only, and changing any level clears every level beneath it - so a stale district left over from a different province can never be submitted.
- c. Wired into the public application form (Province, then District, then Sector) and the student profile page (full five-level hierarchy). "Residence District" is now a validated dropdown rather than free text.

**Verification.** Dataset verified programmatically: exactly 5 provinces and 30 districts (3 + 5 + 8 + 7 + 7), no duplicates, correct province mapping, and safe fallbacks for unknown input.

**Note.** Scope decision, agreed in advance: Sector, Cell and Village remain free text. Rwanda has 416 sectors, 2,148 cells and roughly 14,837 villages, and shipping an approximated list would be worse than free text - a dropdown missing a user's real sector actively prevents them entering the truth. LocationSelect converts a level to a dropdown automatically as soon as the official RGB/NISR dataset is loaded into **rwandaLocations.ts**. No code change is required to enable it.

### 3. Notification feature in header doesn't work

#### REPORTED

The Notification icon did not respond to clicks. Unlike the Message icon, which opens a dropdown of recent messages, Notifications produced no visible response.

#### ROOT CAUSE

Two separate gaps. The header button was a decorative element with a hardcoded unread dot and **no click handler at all**. Behind it, notification rows were being written to the database (overdue-fee alerts, for example) but **no endpoint ever read them back**, so even a working button would have had nothing to display.

#### WHAT WAS CHANGED

- a. Built the missing read API: **GET /api/notifications** (recent items plus a true unread count), **POST /api/notifications/:id/read**, and **POST /api/notifications/read-all**.
- b. Every endpoint is scoped to the signed-in user in the SQL WHERE clause, so one user can never read or dismiss another's notifications.
- c. Replaced the dead button with a working **NotificationBell**: a real unread badge, a dropdown of recent notifications, click-through to the relevant page, mark-one/mark-all-read, and dismissal by outside-click or Escape.
- d. Added an explicit "You have no notifications" empty state - the original complaint was silence, so the component must visibly confirm it is working even when there is nothing to show.

**Verification.** 14 of 14 automated checks passed, including an access-control probe confirming that attempting to mark another user's notification as read leaves that row untouched, and that unauthenticated requests are rejected with 401.

## 4. During Application process users can skip required documents

### REPORTED

Applicants could click Next on the Required Documents step without uploading anything, skipping the step entirely.

### ROOT CAUSE

The wizard validates each step against a map of form fields. The Documents step has no form fields, so it had **no entry in that map** - and the validation routine simply waved through any step it found no entry for.

### WHAT WAS CHANGED

- a. The Documents step is now gated on the faculty's requirements checklist rather than on form fields. The Continue button is disabled while anything mandatory is outstanding, and names exactly which documents are missing.
- b. The same gate is applied to the step-progress rail, so the block cannot be walked around by jumping steps.
- c. Added a matching server-side check in the submit endpoint. A client-side check is a courtesy, not a control - without the server gate an applicant could post directly to the API and submit with nothing attached.
- d. A faculty with no requirements configured is deliberately not blocked, so an unconfigured checklist never becomes an unpassable gate.

**Verification.** Gate logic shares one cached data source with the list the applicant sees, so the block and the on-screen checklist can never disagree. Type-checked and built clean.

## 5. Changing password within student portal sends an internal server error

### REPORTED

Attempting to change a password showed "An internal server error occurred", with a 500 (Internal Server Error) in the console.

### ROOT CAUSE

The password-change routine loaded the user with **find()**, which passes every row through a field-hiding step that deliberately **strips the password hash**. The code then passed that now-missing value into **password\_verify()**, which raised a fatal `TypeError`. The feature could never have worked for any user.

### WHAT WAS CHANGED

- a. Switched to **findBy('id', ...)**, which returns the raw row including the hash - the method that already exists in the codebase precisely for password and token verification.
- b. Added a clear response for accounts with no password set, pointing the user to "Forgot password" rather than failing obscurely.
- c. Audited every other site that reads a hidden field; all were already using the correct method, so the defect was isolated to this one function.

**Verification.** Reproduced the exact fatal error first, then confirmed the fix end to end: the correct current password updates and persists the new hash; a wrong one returns 400; reusing the same password is refused; an unknown user returns 404. The test restored the original hash afterwards.

## 6. Uploading National ID or Passport fails

### REPORTED

Submitting a service request returned "Invalid file type for 'Copy of National ID or Passport'. Allowed: pdf, jpg, jpeg, png" with a 422 response - despite the file being in one of those very formats.

### ROOT CAUSE

The service catalogue stores permitted types as **file extensions** ("pdf", "jpg", "jpeg", "png"), but the validation code converted the uploaded file to a **MIME type** ("application/pdf") and compared that against the extension list. The two can never match, so **every** upload was rejected - by an error message that listed the exact formats it was refusing.

### WHAT WAS CHANGED

- a. Both sides of the comparison are now normalised to canonical extensions before matching, so the check works whether an administrator authored the catalogue entry as extensions or as MIME types.
- b. Handles real-world variations: uppercase extensions, the "jpeg"/"jpg" spelling difference, the non-standard *image/jpg* some browsers send, files with no extension, and files with no reported type.
- c. Content-based verification still runs afterwards, so a renamed executable is caught regardless of what its filename claims.

**Verification.** 15 of 15 cases passed, driven by the actual catalogue rows from the database - including the precise combination that was failing in production. Genuinely invalid files (.exe, .xlsx) are still correctly rejected.

## 7. Fine payment

### REPORTED

A fine raised a student's total invoiced amount, but there was no way to view or pay it. My Finance did not reflect the updated amount shown elsewhere in the profile.

### ROOT CAUSE

Two compounding faults. First, when a fine is issued the system creates an invoice for it and stamps an academic year chosen by **highest ID** rather than the year flagged as current - which selected a catch-all "Legacy" year instead of 2025/2026, filing the invoice under a year My Finance never queries. Second, **every fines endpoint required staff permissions**, so a student had no way to see their own fines at all.

### WHAT WAS CHANGED

- a. Fines are now filed against the academic year explicitly flagged as current, with a dated fallback if none is flagged, so the invoice lands where the student will actually see it.
- b. Added a student self-service endpoint **GET /api/finance/my/fines**, returning each fine with its reason, amount, status, due date and the invoice it is billed on.
- c. Added a Fines section to My Finance showing why each fine was raised, what is still outstanding, and a Pay button for the total owed.
- d. This section is deliberately not filtered by the academic-year selector: an unpaid fine is owed regardless of which year the student is browsing, and year-filtering is how it became invisible in the first place.

**Verification.** Confirmed against live data that the previous logic selected the "Legacy" year while the corrected logic selects 2025/2026. 7 of 7 tests passed covering outstanding versus waived fines, partial payments, and invoice linkage.

## 8. Fake payment calendar / unclear calendar

### REPORTED

The Payment Calendar showed dates that appeared outdated - a Registration payment dated 01/09/2025 while the current academic period is 2026 - with multiple entries and no way to tell which were current.

### ROOT CAUSE

The dates were **not wrong**. Academic year 2025/2026 legitimately runs from 1 September 2025 to 31 August 2026, so a September 2025 registration deadline is correct. The real defect was presentational: the calendar showed a bare list of dates with no academic year stated, no payment type spelled out, and no indication of which entries still required action - so correct data read as stale data.

### WHAT WAS CHANGED

- a. The academic year is now stated explicitly beneath the heading, which directly answers "why does 2026 show 2025 dates?".
- b. Entries are split into upcoming items and a collapsed **Payment history** section for deadlines that have passed, exactly as proposed in the report.
- c. Each row carries a status chip - Overdue, Due soon (within 14 days), Upcoming or Passed.
- d. Each row now names the payment type ("Registration Deadline", "Installment Due"), plus intake and level where recorded, so entries are identifiable at a glance.

**Verification.** Type-checked and built clean. Status boundaries are computed at day resolution so an item due today is never mislabelled as overdue.

## 9. Application payment bypass (transcript not visible to admin)

### REPORTED

Applicants who upload proof of payment instead of paying can submit their application, but administrators could not view the submitted document on the Admin Admission page - so they could reject an application without ever reviewing the evidence.

### ROOT CAUSE

The document was uploaded to storage successfully and its reference returned to the browser - but **never written to the application record**. The identifier was discarded the moment the response was sent, so the file existed in storage with nothing pointing to it. The admin viewing screen already existed and worked correctly; it simply had nothing to display.

### WHAT WAS CHANGED

- a. The uploaded document reference and its file type are now persisted to the application record, in the fields the existing admin document viewer already reads.
- b. A failed upload now fails the request instead of silently recording the payment as verified. This document is the only evidence behind a manually declared payment, so recording the claim while losing the proof left admins deciding blind - exactly the situation reported.

**Verification.** The admin-side download endpoint and the "view proof" control on the Admin Admission page already existed and were confirmed to read these fields, so populating them restores the full review path.

## Findings Requiring Your Attention

---

The following were discovered while investigating the reported issues. They were deliberately **not** changed, either because they require an operator decision or because they fall outside the scope of this report. Each is recorded here so it is not lost.

### Storage service URL is wrong in the local backend configuration

backend/.env points FILE\_SERVER\_URL at port 8888, but the web server serves port 80. The new diagnostic GET /api/health?deep=1 currently reports the storage service as configured but not reachable. **Run this same check on the production server** to confirm issue 1 is genuinely resolved there - the 401 originally reported means the deployed FILE\_SERVER\_KEY does not match the one in file-server/.env, which can only be corrected on the server itself.

### **The database is missing primary keys and auto-increment**

The `users`, `notifications`, `fee_fines` and `fee_invoices` tables have no PRIMARY KEY and no AUTO\_INCREMENT. The effects are already visible in live data: duplicate user IDs exist, and an applicant who registered on 17 August was assigned ID 0. **New registrations will keep colliding until these constraints are restored.** This was left untouched because altering the database schema is an operator decision, not a side effect of a bug-fix pass.

### **Fine creation can fail when the issuing user cannot be resolved**

`fee_invoices.created_by` is NOT NULL, but the fine-creation routine passes null when it cannot resolve the acting user. Reported rather than changed, as it sits outside the nine issues in this report.

### **Pre-existing type errors in the Offers page**

Three TypeScript errors in `frontend/src/pages/admin/admissions/OffersPage.tsx` (lines 206, 215 and 274) predate this work, originating in commit 33ffdab. They were left untouched and are unrelated to the reported issues.

### **The same academic-year fault exists elsewhere**

`GateManagementController` and `DeliberationController` select the academic year by highest ID, the same pattern corrected in issue 7. These were left alone because they fall outside the Student Portal scope of this report, but they will misbehave in the same way.

---

All changes type-check and compile cleanly, and the application builds for production. Automated checks: 14/14 notifications (including access-control probes), 15/15 file-type validation against live catalogue data, 7/7 fines, plus full password-change and file-storage failure-mode coverage.